```
Anagrams of teaching: cheating, teaching
Anagrams of ruby: bury, ruby
```

If we were typing in this program for casual use, we might just enter it into a single file (perhaps anagram.rb). It would look something like this:

gems/anagram.rb

```
#!/usr/bin/env ruby

require 'optparse'

dictionary = "/usr/share/dict/words"

OptionParser.new do |opts|

  opts.banner = "Usage:  anagram [ options ]  word..."

  opts.on("-d", "--dict path", String, "Path to dictionary") do |dict|
    dictionary = dict
  end

  opts.on("-h", "--help", "Show this message") do
    puts opts
    exit
  end

  begin
    ARGV << "-h" if ARGV.empty?
    opts.parse!(ARGV)
  rescue OptionParser::ParseError => e
    STDERR.puts e.message, "\n", opts
    exit(-1)
  end
end

# convert "wombat" into "abmotw". All anagrams share a signature
def signature_of(word)
  word.unpack("c*").sort.pack("c*")
end

signatures = Hash.new

File.foreach(dictionary) do |line|
  word = line.chomp
  signature = signature_of(word)
  (signatures[signature] ||= []) << word
end

ARGV.each do |word|
  signature = signature_of(word)
  if signatures[signature]
    puts "Anagrams of #{word}: #{signatures[signature].join(', ')}"
  else
    puts "No anagrams of #{word} in #{dictionary}"
  end
end
```

You might be wondering about the line word.unpack("c*").sort.pack("c*"). This uses the function unpack to break a string into an array of characters, which are then sorted and packed back into a string.